

Optimising Network Packet Processing on Low Power ARM Single-Board Computers via DPDK and XDP

Aditya Choubey



Master of Science
Computer Science
School of Informatics
University of Edinburgh
2026

Abstract

Contemporary research has established that clusters of low-power, "wimpy" ARM-based single-board computers (SBCs) are a viable architecture for edge computing and can provide performance comparable to conventional infrastructure at considerably lower cost and energy consumption. As this architecture matures from feasibility studies towards practical deployment, its scalability increasingly relies on the efficiency of inter-node communication. Yet, the conventional Linux networking stack imposes fixed per-packet costs that consume a disproportionate share of the limited processing capacity available on such constrained hardware. Kernel-bypass techniques such as Data Plane Development Kit (DPDK) and eXpress Data Path (XDP) address this problem by avoiding most or all of the kernel's involvement in packet processing, and are well established on server-class x86 systems. Their behaviour on low-power ARM single-board computers however, remains comparatively unexplored, primarily due to the driver- and hardware-level support these techniques require has previously targeted servers rather than the embedded Ethernet controllers found on ARM SBCs.

This dissertation explores the extent to which native DPDK and XDP support can reduce communication overhead and improve packet-processing performance on low-power ARM hardware, using the Raspberry Pi 5 as our platform for evaluation. Its onboard Ethernet controller lacked both native XDP support in its Linux driver and a DPDK poll-mode driver. Hence, the primary contribution of this work is to extend the network driver with native XDP support, closing this broader gap. A network benchmarking suite was developed to compare this accelerated path against conventional TCP and UDP sockets in terms of round-trip latency, driver overhead, throughput, and packet-processing rate. As a secondary, exploratory contribution, a prototype native DPDK driver and an accompanying cache-synchronisation mechanism for non-coherent ARM memory architectures were also developed, laying essential groundwork for complete kernel bypass on similar platforms. The results demonstrate that kernel-bypass networking, long considered to be a server-class technique, can be extended to low-power ARM single-board computers, reducing latency and driver-level overhead relative to conventional sockets while revealing where its throughput benefits and current limitations lie. These findings inform the design of future networking paths for edge-computing hardware built on similarly constrained ARM platforms.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Aditya Choubey)

Acknowledgements

I would like to thank my supervisor, Dr. Yuvraj Patel, for his guidance and mentorship, not only throughout the dissertation but also during my entire academic journey at The University of Edinburgh.

I would also like to thank Rachel Somerset for acting as my unofficial mentor for the dissertation and providing invaluable advice.

Additionally, I would also like to express my gratitude to my parents and my sister for their unwavering support throughout my life and especially during my academic journey. Finally, I would like to thank my partner, Janvi Dhanani, for her endless patience, love, and support through every stage of this journey.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Contributions	3
2	Background	5
2.1	Edge Networks	5
2.2	Wimpy Node Clusters	6
2.3	Linux TCP/IP Network Stack	8
2.4	Distributed Scheduling	9
2.5	Kernel Bypassing	10
2.6	Research Gaps	12
3	Design	14
3.1	Overview	14
3.2	Existing Mechanisms and Hardware Support	14
3.3	Limitations and Design Constraints	15
3.4	Design Goals	17
3.5	Primary Design: Native AF_XDP Support for the Cadence GEM Driver	17
3.6	Exploratory Design: Native DPDK PMD for Cadence GEM Block . .	19
3.7	Summary	21
4	Implementation	22
4.1	Development Environment	22
4.2	Adding Native XDP and AF_XDP Support in the Network Driver . . .	22
4.3	Alternative DPDK PCI Poll-Mode Driver	27

5	Evaluation	30
5.1	Experimental Setup	30
5.2	Round-Trip Latency (RTT) Performance	30
5.3	Driver Overhead	32
5.4	Throughput Performance	33
5.5	Packet-Processing Rate	35
5.6	Discussion	36
5.7	Limitations	37
6	Conclusion	39
6.1	Future Research Directions	39
6.2	Final Remarks	40
	Bibliography	41
7	Appendix	44
7.1	Algorithms	44
7.2	Raw Benchmark Results	49

Chapter 1

Introduction

1.1 Motivation

Distributed computing has traditionally relied on “brawny” infrastructure, such as high-performance servers, Graphics Processing Units (GPUs), and specialised accelerators [1]. The financial and energy costs of such systems have motivated interest in clusters of low-power “wimpy nodes” such as Raspberry Pi single-board computers [12]. These platforms are particularly relevant to edge computing, where computation is placed close to data sources to reduce latency and back-haul traffic. Although wimpy-node clusters can provide favourable cost and energy efficiency, their scalability increasingly depends on efficient communication between nodes.

Inter-node communication commonly uses the conventional Linux networking stack, which provides mature protocol functionality but also introduces per-packet costs through system calls, memory copies, socket-buffer management, protocol processing, and scheduling. These costs consume a proportionally larger share of the available processing capacity on low-power ARM systems. Kernel-bypass and accelerated packet-processing technologies, including the Data Plane Development Kit (DPDK) and eXpress Data Path (XDP), can shorten this path through early driver processing, shared-memory packet exchange, polling, and batching. However, their benefits depend on hardware and driver support, which was previously absent for the Raspberry Pi 5’s onboard Cadence GEM Ethernet controller.

1.2 Problem Statement

The conventional Linux networking stack was designed for general-purpose use, and its per-packet costs are increasingly difficult to justify on low-power ARM hardware, where CPU resources are already scarce relative to the demands of distributed and edge workloads. Kernel-bypass techniques such as XDP and DPDK exist precisely to remove this overhead from the critical packet-processing path, and have proven their value on server-class systems [10, 18]. Realising these same benefits on low-power ARM single-board computers, however, is not simply a matter of applying existing tools. Kernel bypass fundamentally depends on driver- and hardware-level support that has primarily been developed for server-class Network Interface Cards (NICs), and extending it to embedded ARM Ethernet controllers requires reworking this support from within the driver itself. This task is further complicated on ARM platforms by non-coherent DMA architectures, which require explicit cache maintenance whenever packet buffers change ownership between the CPU and the network device, a concern that does not arise in the same way on coherent x86 systems.

This leaves both a practical and an empirical problem. Practically, low-power ARM platforms cannot benefit from kernel-bypass networking until the necessary driver support is built for them, rather than assumed to transfer directly from server hardware. Empirically, it remains uncertain whether the benefits of shortening the kernel networking path, well established on server-class systems, actually materialise on constrained ARM hardware, where the balance between fixed per-packet costs and available processing capacity is fundamentally different. This dissertation thus asks:

To what extent can native XDP support reduce networking overhead and improve packet-processing performance on low-power ARM single-board computers, compared with conventional Linux TCP and UDP sockets?

The work focuses on establishing and evaluating this networking path itself, as this is a necessary prerequisite for its later integration into edge-computing and distributed workloads.

1.3 Objectives

The primary objective of this dissertation is to determine whether accelerated packet processing can reduce networking overhead on the Raspberry Pi 5. This is addressed through the following objectives:

1. Extend the Raspberry Pi 5's Cadence GEM (`macb`) driver with native XDP support and enable an accelerated AF_XDP userspace datapath.
2. Develop a common benchmark suite for comparing conventional TCP and UDP sockets with DPDK via AF_XDP.
3. Evaluate the approaches in terms of round-trip latency, driver overhead, throughput, and packet-processing rate across a range of payload sizes.
4. Investigate the feasibility of complete kernel bypass through a prototype native DPDK PMD and an ARM cache-synchronisation mechanism.

1.4 Contributions

This dissertation makes four principal contributions:

1. **Native XDP and AF_XDP support for the Raspberry Pi 5 Ethernet interface.** The Linux `macb` driver was extended to execute XDP programs within its receive path, support the principal XDP actions, and redirect packets to AF_XDP userspace applications. This enables accelerated packet processing on hardware for which no driver-native solution previously existed.
2. **A comparative evaluation on low-power ARM hardware.** A common C benchmark suite was developed to compare TCP, UDP, and DPDK through AF_XDP on identical Raspberry Pi 5 systems. The evaluation measures latency, tail latency, driver overhead, throughput, and packet-processing rate across multiple payload sizes.
3. **Empirical evidence of the benefits and trade-offs of accelerated networking.** The AF_XDP-based path reduced median and tail RTT and substantially lowered measured driver overhead compared with conventional socket networking. It also achieved the highest throughput for large packets, although its small-packet throughput remained slightly below UDP. The results therefore identify both the potential and the current limitations of accelerated networking on constrained ARM hardware.
4. **An exploratory path towards complete kernel bypass.** A prototype native DPDK PMD and supporting cache-synchronisation module were developed

for the Raspberry Pi 5's non-coherent DMA environment. Although not fully evaluated, this work provides a foundation for future zero-copy packet processing on the platform.

Chapter 2

Background

2.1 Edge Networks

The surge of smart devices, IoT sensors, and latency-sensitive applications (like augmented/virtual reality, autonomous vehicles, and real-time video analytics) has driven network traffic to volumes that conventional centralised data center architectures struggle to accommodate. According to Wang *et al.* [19], mobile data traffic increased approximately 4000 times in the decade prior to their survey, with video traffic being responsible for more than half of the total amount and still growing. In such a centralised architecture, every request must traverse the backhaul to a data centre, producing escalating latency as round-trip distance dominates response time, and backhaul congestion as duplicate traffic for popular content and computation is repeatedly re-transmitted across the same links.

Edge computing was proposed as a structural response: rather than centralising computation, storage, and content in the cloud, these resources are pushed outward to the network’s edge (base stations, gateways, routers, and end-user devices) so that processing happens physically closer to where data is generated. Wang *et al.* [19] define this architecture as one that “deploys and utilizes flexible computing and storage resources at the mobile network edge... with the help of Software Defined Networks (SDNs) and Network Function Virtualisation (NFV),” and group its resources into three categories: *computing*, *caching*, and *communication*. This general idea has been implemented through several concrete proposals [19]: **Mobile Edge Computing (MEC)**, which deploys virtualised servers directly at cellular infrastructure (e.g. the eNodeB); **Fog computing**, which extends the cloud–edge spectrum through a distributed hierarchy of “fog nodes” aimed at latency- and mobility-sensitive IoT workloads;

Cloudlets, self-contained “data centres in a box” deployed at WiFi or cellular access points; and **Edge caching**, which places content storage at base stations and end-user devices to avoid repeated retrieval from distant Content Delivery Networks (CDNs). Across all four, the reported benefits are consistent: reduced latency from physical proximity, reduced backhaul bandwidth consumption (up to 67% in some studies), improved energy efficiency, and the ability to exploit local context that would otherwise be unavailable at a distant cloud [19].

Relevance to wimpy-node infrastructure. These architectures are consistently described as running on constrained, distributed hardware — small base stations, gateways, and, increasingly, low-power single-board computers — rather than data-centre-class servers. These characteristics render the Raspberry Pi and other ARM-based Single Board Computers (SBCs) a viable physical platform for edge deployments. In contrast to typical server hardware, they are inexpensive, low-power, and able to be distributed widely throughout a network. However, the edge-computing literature surveyed here treats the *placement* of computing and caching resources as the primary design problem, largely abstracting away the *cost of moving data between edge nodes* once that placement is decided. Communication is considered as a resource to be allocated and scheduled [19] rather than as a bottleneck arising from how packets are actually processed by the host’s network stack. This is exactly the gap this dissertation is concerned with. Even if computing and caching are optimally placed at the edge, the per-node communication path — kernel-mediated sockets, memory copies, interrupt handling — determines how much of that theoretical benefit is actually realisable on low-power hardware. Section 2.5 returns to this problem in detail, examining where conventional kernel networking introduces overhead and how kernel-bypass techniques attempt to remove it.

2.2 Wimpy Node Clusters

The use of low-power (“wimpy”) nodes for distributed computing has emerged as a promising alternative to traditional high-performance clusters, particularly in terms of energy efficiency. The **FAWN (Fast Array of Wimpy Nodes)** architecture demonstrates that clusters composed of low-power embedded processors coupled with lightweight storage can achieve orders-of-magnitude improvements in energy efficiency for data-intensive workloads [1, 2]. FAWN’s central observation is that many large-scale ap-

plications are I/O-bound rather than compute-bound, and therefore benefit more from architectures that balance computation, storage, and networking capability than from architectures built around raw computational performance. However, FAWN's evaluation is centred on key-value storage workloads, and does not address compute-intensive distributed workloads such as edge computing, matrix multiplication, or other tightly-coupled parallel computation.

Extension to edge computing. Subsequent work extends these ideas from storage-oriented clusters to edge computing environments, where low-power ARM-based systems are deployed physically closer to data sources. Loghin *et al.* [12] analyse the time, energy, and cost trade-offs between wimpy edge systems and traditional cloud infrastructure. Their results show that clusters of low-power devices can achieve performance comparable to high-end cloud instances when scaled appropriately, while offering substantial improvements in cost and energy efficiency. Notably, their study includes a distributed matrix multiplication workload implemented using MapReduce [6], providing evidence that wimpy clusters are applicable to compute-intensive tasks and not only to I/O-bound storage workloads as in FAWN. More recently, Somerset and Patel [17] demonstrate this potential directly using a cluster of ten Raspberry Pi 5 devices for distributed matrix multiplication. Their evaluation explicitly considers the effects of network latency and workload balancing, highlighting communication as an important consideration when scaling wimpy-node clusters. They report that the resulting cluster can provide comparable performance to a conventional desktop machine, reinforcing the viability of low-power ARM clusters for distributed edge computing.

The widening CPU-I/O gap. The growing importance of communication efficiency in distributed systems is further underscored by the widening gap between CPU and I/O performance, in which overall system throughput is increasingly constrained by data movement rather than by raw computation [1]. This issue is particularly pronounced on low-power ARM clusters. Somerset and Patel [17] similarly identify network latency and communication overhead as important considerations when scaling distributed workloads across Raspberry Pi 5 nodes. The CPUs are already resource-constrained relative to server-class hardware, so the fixed per-packet costs of kernel-mediated networking (which includes system calls, interrupts, context switches, etc.) consume a proportionally larger share of the available compute budget than they would on more

powerful nodes. Taken together, these observations suggest that the communication layer represents an important obstacle to scaling compute-intensive workloads on wimpy-node clusters, motivating the focus of this dissertation on the networking path itself rather than on higher-level distributed-computing frameworks.

2.3 Linux TCP/IP Network Stack

The conventional networking path in Linux connects an application-level socket to the network interface controller (NIC) through a tightly integrated sequence of kernel subsystems. Cai *et al.* [4] characterise this path as a single end-to-end pipeline comprising userspace/kernel data copies, TCP/IP protocol processing, socket-buffer (`sk_buff`) management, memory allocation, network-device and driver processing, interrupt handling, and CPU scheduling, and study the contribution of each stage to overall CPU cost. Their central finding is that no single stage dominates in general: the cost of conventional networking emerges from the cumulative, tightly coupled overhead of several stages rather than from one bottleneck that can be optimised away in isolation.

On the transmit path, an application's write is copied into kernel-managed `sk_buffs`, processed through the TCP/IP stack, and handed to the NIC driver for DMA transfer, with mechanisms such as Generic/TCP Segmentation Offload (GSO/TSO) reducing the number of packets that require full per-packet processing. The receive path is largely symmetric but is complicated by its asynchronous origin at the NIC: incoming frames are DMA'd into a kernel-managed page pool, batched and polled via NAPI to amortise interrupt overhead, optionally aggregated through Generic Receive Offload (GRO), and steered to a CPU core via mechanisms such as RSS, RPS, and RFS before being copied into the application's userspace buffer on `read()/recv()` [4].

Critically, Cai *et al.* show that the relative importance of these stages shifts with link speed: at 100 Gbps, a single CPU core could not sustain line-rate processing even with standard NIC and kernel optimisations, and receiver-side data copying alone could account for more than half of the CPU cycles consumed by a single long-lived flow [4]. They further show that CPU/NUMA locality has a substantial effect on this cost, since data that is not consumed promptly can be evicted from cache before the application reads it.

These findings motivate the focus of this dissertation on low-power ARM hardware specifically. The fixed, per-packet costs identified by Cai *et al.* (buffer management, interrupt handling, memory copies, context switches) are not unique to high-bandwidth

server links; they are equally present at lower packet rates, but on a constrained CPU they consume a proportionally larger share of the available compute budget than on server-class hardware. This provides the immediate motivation for examining approaches that shorten or bypass portions of this path, including DPDK and AF_XDP, discussed in Section 2.5.

2.4 Distributed Scheduling

Efficient distributed execution depends not only on computational parallelism, but also on the ability of the underlying system to schedule tasks and manage communication with minimal overhead. Prior work has approached this from three complementary directions: decentralised scheduling, lightweight edge orchestration, and more recently, scheduling systems built directly on kernel-bypass networking.

Sparrow [15] replaces centralised cluster scheduling with multiple stateless schedulers that place tasks using randomised, “power of two choices”-style sampling, combined with *batch sampling* and *late binding* to control tail latency without globally synchronised state. It reports response times within 12% of an idealised omniscient scheduler while retaining high throughput and fault tolerance, establishing that decentralised scheduling can scale to short-lived, high-throughput task workloads.

At the edge, Oakestra [3] addresses the poor fit of data-centre orchestrators such as Kubernetes for heterogeneous, geographically distributed, and unreliable infrastructure. It introduces a three-tier root/cluster/worker hierarchy with delegated scheduling and a semantic overlay network, and reports roughly 20% faster deployment than K3s under network delay and 50–60% faster deployment under 20–50% packet loss [3], evidence that control-plane design choices along with data-plane throughput, become dominant once network conditions degrade. Follow-up work has begun benchmarking lightweight Kubernetes distributions (k0s, k3s, KubeEdge, OpenYurt) directly on Raspberry Pi-class hardware [8], but this evaluation remains confined to control-plane cost (CPU, memory, deployment latency) rather than the underlying data-plane communication path.

A parallel line of work explores scheduling directly on top of kernel-bypass networking, treating it as a prerequisite for microsecond-scale decisions rather than an independent concern. Shenango [14] reassigns CPU cores across applications every 5 μ s using a dedicated kernel-bypass I/O layer. Caladan [9] extends this with a centralised scheduler and kernel-bypass networking (via `libibverbs` or DPDK) to isolate latency-critical tasks from interference at the microsecond timescale. Shinjuku [11]

achieves similar objectives through centralised dispatch with low-overhead preemption. All three treat conventional kernel-induced networking overhead as incompatible with fine-grained scheduling, reasserting the premise of this dissertation.

Taken together, this body of work establishes that decentralised scheduling, hierarchical orchestration, and kernel-bypass-native scheduling all treat communication cost as a first-class design constraint. However, the systems most directly relevant to this dissertation’s aims, Shenango [14] and Caladan [9], have only been demonstrated on server-class x86 hardware with mature kernel-bypass NIC support, while orchestration work targeting Raspberry Pi-class hardware [8] evaluates the control plane only. No existing work combines a scheduling or orchestration layer for wimpy ARM clusters with a validated kernel-bypass data plane for the underlying NIC — a gap this dissertation addresses directly in Section 2.5 and the chapters that follow.

2.5 Kernel Bypassing

Section 2.3 showed that conventional Linux networking distributes packet-processing cost across several tightly coupled kernel stages (data copies, protocol processing, buffer management, interrupts, scheduling) and that these fixed per-packet costs consume an inordinate share of the available compute budget on constrained CPUs. **Kernel bypass** refers to a family of networking techniques that reduce or completely eliminate the involvement of this conventional stack from the application’s packet-processing path, instead providing a more direct interface to the NIC and its packet buffers. As surveyed by Chen *et al.* [5], kernel-bypass approaches can broadly be divided into user-space packet I/O frameworks, user-space TCP/IP stacks, and hybrid approaches that retain selected kernel functionality.

The objective of kernel bypass is not simply to move networking into userspace, but to remove specific, identifiable sources of overhead from the conventional data path: system calls issued per packet, userspace/kernel data copies, interrupt-driven dispatch, buffer allocation and management, and poor control over CPU/memory locality. Different kernel-bypass designs remove different subsets of this overhead, and hence make different trade-offs between performance, programmability, and kernel integration. Two approaches are of relevance to this dissertation: the Data Plane Development Kit (DPDK), which provides a predominantly userspace packet-processing environment, and the eXpress Data Path (XDP)/AF_XDP, which retains the kernel as the primary networking environment while providing an accelerated path to userspace.

Data Plane Development Kit

Originally introduced by Intel and later developed as an open-source project, DPDK is one of the most pervasive frameworks for high-performance userspace packet processing. It provides libraries, memory-management facilities, and poll-mode drivers (PMDs) that enable an application to interact with supported NICs entirely outside the conventional Linux network path [10].

Instead of depending on interrupts, a DPDK application manages a dedicated thread to continuously poll the NIC's receive and transmit queues, removing interrupt-handling overhead from the critical path and enabling batched processing with explicit control over CPU/cache locality. Packet buffers are drawn from large, pre-allocated hugepage-backed memory pools rather than the conventional kernel allocator, and are exchanged via lockless ring structures and batch-oriented APIs. Together, these mechanisms replace the conventional *Application* → *Socket API* → *TCP/IP stack* → *netdevice* → *driver* → *NIC* path with direct interaction between a userspace PMD and pre-allocated buffers, significantly decreasing per-packet overhead. Modi and Kulkarni report that DPDK-based userspace networking achieves higher throughput and lower latency than conventional kernel TCP/IP networking under HPC workloads [13], a result directly relevant to distributed applications where inter-node communication can become a limiting factor as parallelism increases.

This performance, however, comes at the cost of reduced integration with the conventional networking model. Applications must explicitly manage packet buffers, polling threads, CPU affinity, and protocol functionality otherwise handled by the kernel, and native operation depends on the availability of a suitable PMD for the target NIC. While DPDK is mature on server-class x86 hardware, its behaviour on resource-constrained ARM systems, where such PMD support is far less mature, remains comparatively unexplored.

XDP and AF_XDP

An alternative approach removes only selected stages of the conventional path while retaining the Linux kernel as the primary networking environment. XDP allows an eBPF program to execute at a very early point in the receive path, within the network driver itself, where a packet can be inspected, dropped, passed to the normal stack, transmitted back out, or redirected — bypassing a significant portion of the conventional *sk_buff*-based processing pipeline described in Section 2.3.

AF_XDP extends this mechanism with a socket-based interface: an XDP program redirects a packet into an `XSKMAP`, from which it is delivered to an AF_XDP socket and an associated userspace memory region (UMEM), with kernel/userspace communication organised through four lockless rings (RX, TX, fill, and completion) [18]. This gives applications a substantially shorter path to the NIC than conventional sockets, while the kernel retains control over device management and configuration: a different point in the design space from DPDK’s largely kernel-independent model.

The performance benefit AF_XDP can realise depends critically on driver support. A generic `XDP_SKB` mode copies packets from the kernel’s representation into UMEM, whereas a driver-native `XDP_DRV` mode provides a more direct path, and only where hardware and driver support it can AF_XDP operate in true zero-copy mode, eliminating the kernel-to-UMEM copy entirely [18]. AF_XDP therefore does not itself guarantee zero-copy operation—this is determined entirely by the capabilities of the underlying NIC driver.

This driver dependency is directly significant to this dissertation. The Raspberry Pi 5’s onboard Ethernet controller is served by the Cadence GEM (`macb`) driver, which, prior to this work, provided no native XDP support and therefore could not expose an accelerated AF_XDP path; the same limitation also prevents effective use of DPDK’s `af_xdp` PMD, which itself depends on the underlying Linux driver’s AF_XDP capabilities [7]. DPDK and AF_XDP thus address the same underlying limitations of conventional networking (per-packet kernel processing, interrupt overhead, unnecessary data movement, and poor processing locality) but from different points in the system architecture, and on this platform neither has, prior to this work, been able to realise its intended benefits.

2.6 Research Gaps

Although kernel-bypass networking has been extensively studied in server-class environments, its behaviour on low-power ARM systems remains less well understood. This is particularly important for wimpy-node clusters, where the CPU resources available for packet processing are substantially more constrained than those of conventional data-centre servers. Simpson *et al.*, for example, evaluate an XDP-based dataplane on Raspberry Pi-class hardware and demonstrate improvements in latency and consistency over AF_PACKET-based processing, particularly under high packet rates [16]. Their work nevertheless focuses on processing packets within the XDP dataplane rather than

providing a complete AF_XDP userspace networking path.

More importantly, the Raspberry Pi 5 presents a hardware- and driver-specific gap in the existing networking ecosystem. Prior to this work, the Cadence GEM ([macb](#)) driver did not provide native XDP support, preventing the platform from exploiting the accelerated AF_XDP datapath. At the same time, there was no native DPDK PMD providing direct access to this network hardware. Consequently, the performance trade-offs between conventional Linux sockets, AF_XDP, and DPDK could not be evaluated directly on this platform.

This dissertation addresses this gap by implementing native AF_XDP support for the Raspberry Pi 5's network hardware and investigating a complementary DPDK-based packet-processing path. These implementations are evaluated against the conventional Linux TCP/IP stack to determine how reducing the kernel networking path affects throughput, packet-processing rate, latency, and CPU efficiency on low-power ARM hardware.

Chapter 3

Design

3.1 Overview

This chapter outlines the design philosophy, objectives, and architectural decisions made to achieve the objectives of this dissertation. Section 3.2 briefly explains the networking mechanisms and kernel-bypass hardware/driver support available prior to this research effort. Section 3.3 isolates the specific limitations this created for the Raspberry Pi 5's Cadence GEM (`macb`) controller and maps each directly onto a design response. Section 3.4 states the design goals that follow from this analysis, and Sections 3.5 and 3.6 present the two designs developed to meet them: native AF_XDP support integrated into the `macb` driver, and an exploratory native DPDK Poll Mode Driver (PMD) with a custom ARM cache-synchronisation mechanism. Both designs target identical Raspberry Pi 5 hardware and the same Ethernet interface, so that the performance differences measured in Chapter 5 can be attributed to the software path rather than to variation in the underlying hardware.

3.2 Existing Mechanisms and Hardware Support

Conventional Linux Socket Path

The baseline against which both designs are compared is the conventional Linux TCP/IP socket path described in Section 2.3: packets are delivered through interrupt-driven NAPI polling, copied into and out of `sk_buffs`, processed by the kernel's TCP/IP stack, and exchanged with the application through a further copy at the `recv()/send()` boundary. This path is pre-existing and general-purpose; it is not a contribution of this

work and is retained throughout only as the performance baseline in Chapter 5.

Kernel-Bypass Support Prior to This Work

Section 2.5 established that DPDK and AF_XDP are mature on server-class x86 NICs, but that their benefit on any given platform depends entirely on driver-level support: DPDK requires a Poll Mode Driver (PMD) written for the target NIC, and AF_XDP's accelerated (XDP_DRV/zero-copy) mode requires the NIC's kernel driver to implement native XDP hooks. Prior to this work, *neither* existed for the Raspberry Pi 5's Cadence GEM controller. The `macb` driver exposed no XDP hooks of any kind, and no DPDK PMD, native or AF_XDP-backed, was available for this NIC. Both frameworks were therefore entirely unusable on this hardware, in any mode, before this project began.

3.3 Limitations and Design Constraints

Table 3.1 summarises the specific limitations this absence of support created, and the design response each motivates. These limitations, rather than general kernel-bypass theory, are what shaped the two designs presented in Sections 3.5 and 3.6.

Table 3.1: Limitations of prior kernel-bypass support for the Raspberry Pi 5’s Cadence GEM controller, their consequences for the packet-processing path, and the corresponding design responses.

#	Limitation (prior to this work)	Consequence for the Data Path	Design Response
L1	No native XDP hooks in the <code>macb</code> driver; only generic, copy-based <code>XDP_SKB</code> operation is possible.	The generic XDP path requires packet data to be copied into buffers accessible through the <code>AF_XDP</code> socket, preventing a one-copy driver-native path.	Extend <code>macb</code> with native XDP support, enabling a one-copy <code>AF_XDP</code> path (Section 3.5) - the primary contribution.
L2	No DPDK PMD exists for the Cadence GEM/RP1 combination; DPDK cannot address the NIC in any mode.	Packets cannot be processed directly by a DPDK userspace datapath and must instead traverse the kernel-managed networking path.	Implement a native DPDK PCI PMD for the RP1-attached GEM controller from scratch (Section 3.6).
L3	Raspberry Pi 5 / RP1 DMA is not cache-coherent. A PMD that bypasses the kernel also bypasses its existing, correct DMA cache-maintenance handling.	A userspace datapath cannot safely share DMA buffers with the NIC without explicitly maintaining cache consistency between the CPU and device.	Design a dedicated cache-synchronisation kernel module exposing explicit clean/invalidate operations to the userspace PMD (Section 3.6).

L1 and L2 are both consequences of driver incompatibility, but they admit different responses. L1 can be resolved *within* the existing kernel driver model, by teaching `macb` to speak XDP; because the resulting `AF_XDP` path still relies on the kernel’s page-pool DMA machinery, it inherits that machinery’s existing, correct cache handling and does not itself confront L3. L2, by contrast, requires bypassing the kernel driver entirely, which reintroduces L3 as a first-class design problem. This distinction between one design that extends the kernel’s existing cache-safe DMA path and one that must

reimplement cache safety itself, is why the two designs are presented separately below, with the former as the primary, fully evaluated contribution and the latter as a secondary, exploratory one.

3.4 Design Goals

Both designs share the following goals, derived directly from Table 3.1:

- Provide a working kernel-bypass path for the Cadence GEM controller where none previously existed (L1, L2).
- Minimise the software layers and memory copies traversed by each packet relative to the conventional socket path.
- Replace interrupt-driven reception with polling, to improve processing determinism.
- Preserve an identical hardware pipeline across all three implementations (sockets, AF_XDP, DPDK), so that performance differences measured in Chapter 5 arise from software architecture alone.
- Where DMA is managed outside the kernel, provide explicit, correct cache synchronisation on ARM rather than assuming x86-style coherency (L3).

3.5 Primary Design: Native AF_XDP Support for the Cadence GEM Driver

The dissertation’s primary contribution is extending the `macb` driver with native XDP support so that AF_XDP can operate over the Raspberry Pi 5’s Ethernet interface, resolving L1 directly. Figure 3.1 contrasts this path against the conventional socket path summarised in Section 3.2, classifying each transition as a *copy*, a *move* (ownership transfer without duplication), or a *command* (a control operation such as an interrupt or a poll).

Shared Hardware Pipeline

Both the conventional and AF_XDP paths share an identical hardware datapath. Frames are received by the Ethernet physical layer, processed by the GEM MAC, and trans-

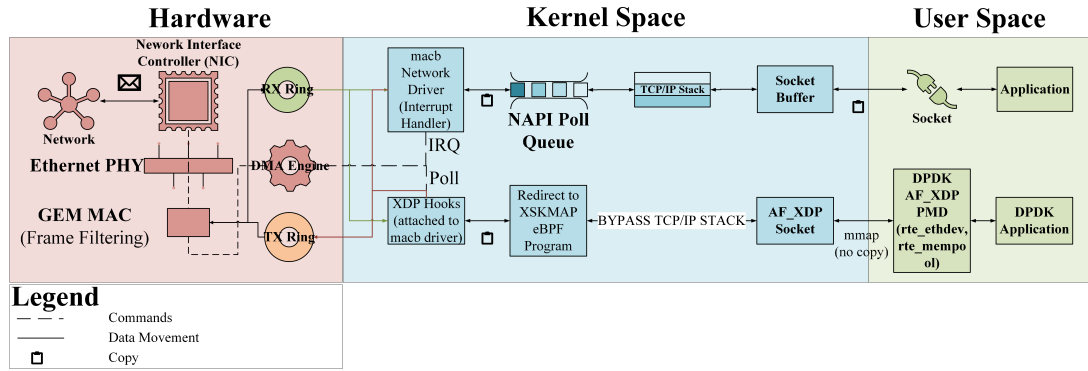


Figure 3.1: Comparison of the conventional Linux packet-processing pipeline (existing baseline) and the proposed AF_XDP path (this work’s primary contribution).

ferred into main memory by the DMA engine through receive descriptor rings, and symmetrically for transmission [4]. This hardware behaviour is common to every design evaluated in this dissertation and is therefore not itself a differentiating factor; the differences lie entirely in how buffer ownership and copying are managed above this layer.

State of the Art: Conventional Receive/Transmit Path

As described in Section 2.3, the conventional path performs two copies on receive (DMA buffer $\rightarrow$ sk_buff, then sk_buff $\rightarrow$ application buffer on `recv()`) and one copy on transmit (application buffer $\rightarrow$ sk_buff on `send()`), interleaved with interrupt handling, NAPI scheduling, and full TCP/IP protocol processing. This path is unmodified by this work and serves purely as the performance baseline in Chapter 5.

Novel Contribution: AF_XDP Receive/Transmit Path

For reception, the modified `macb` driver runs an XDP hook during NAPI polling, before an `sk_buff` is constructed. The attached eBPF program redirects the packet into an XSKMAP; because the driver does not implement native zero-copy UMEM registration, this redirect performs a single copy into a UMEM frame (the only copy on the receive path). From this point on, only descriptor ownership changes hands: the packet’s UMEM descriptor is placed on the AF_XDP receive ring, and the application accesses the packet directly through the memory-mapped UMEM region [18].

Transmission is the reverse: the application writes packets directly into UMEM buffers and submits descriptors to the AF_XDP transmit ring; the driver builds the

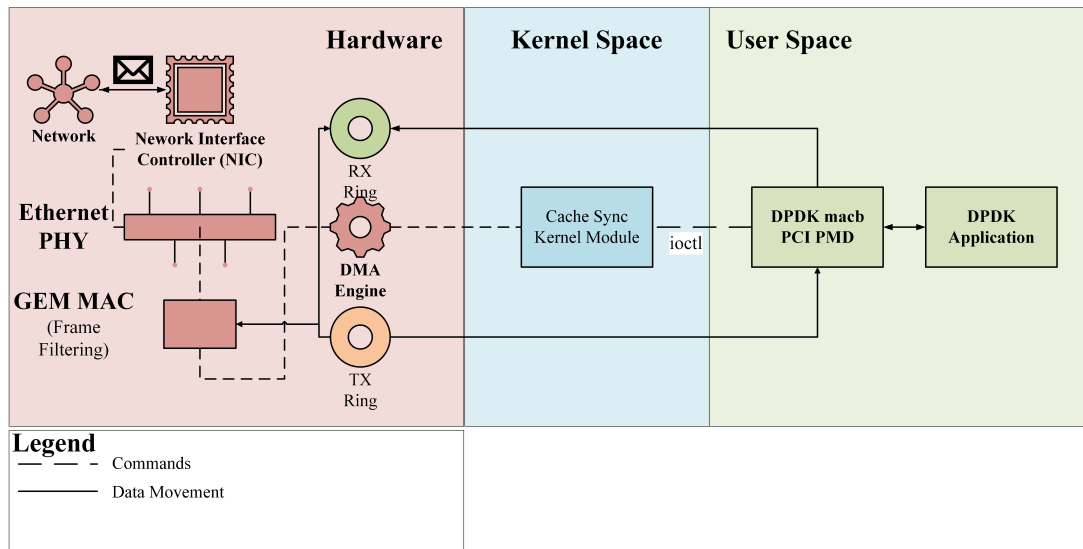


Figure 3.2: Packet-processing flow for the exploratory native DPDK PMD design, including the cache-synchronisation mechanism required by L3.

corresponding DMA transmit descriptors, and the DMA engine reads the packet straight out of UMEM. No copy occurs after the application has written into UMEM.

Comparison and Design Trade-offs

Relative to the two-copy receive / one-copy transmit baseline, the AF_XDP design reduces both paths to a single copy (receive) or none (transmit) once packets reach shared memory. It should be emphasised, however, that this is a *one-copy* design, not a fully *zero-copy* one. True zero-copy AF_XDP requires native UMEM registration in the driver, which was beyond the scope of this contribution. That harder problem of eliminating the remaining copy while correctly handling ARM's non-coherent DMA (L3) is instead investigated directly through the native DPDK PMD design in Section 3.6.

3.6 Exploratory Design: Native DPDK PMD for Cadence GEM Block

While the design outlined in Section 3.5 extends the existing kernel driver, this design bypasses it entirely, resolving L2 by implementing a native DPDK PCI PMD for the Cadence GEM controller as exposed through the Raspberry Pi 5's RP1 I/O controller. It is presented as a secondary, exploratory contribution: architecturally complete but

only partially validated (Section 4.3) due to a lack of time, and reported as a concrete basis for future work rather than as an evaluated result in Chapter 5.

Architecture

Figure 3.2 shows the design’s three layers. At the top, a DPDK application polls the PMD’s `rx_burst()`/`tx_burst()` routines directly, with no kernel involvement in the packet path. The PMD itself owns hardware initialisation, PCI BAR mapping, descriptor-ring management, and the association of DMA descriptors with `rte_mbuf` buffers drawn from a pre-allocated `rte_mempool`; once initialised, it operates entirely in polling mode [10]. A minimal kernel stub (`uio-pci-generic` or `vfio-pci`) is retained only to bind the PCI device and expose its BAR regions and DMA permissions to userspace during setup, and plays no role in steady-state packet processing. The detailed register- and descriptor-level implementation of this driver is given in Section 4.3.

Addressing ARM Cache Incoherence (L3)

Because this design bypasses the kernel’s DMA subsystem entirely, it must independently guarantee that the CPU and the GEM DMA engine observe a consistent view of shared packet buffers on the Raspberry Pi 5’s non-coherent ARM platform, a requirement the AF_XDP design in Section 3.5 avoids by construction. This is addressed with a dedicated `cache_mem_sync` kernel module, exposed to the PMD as a character device, which pins the PMD’s hugepage-backed mbuf pools and performs explicit cache clean and invalidate operations (via the kernel’s `dma_sync_single_for_device/_for_cpu` API) before and after each DMA transaction. This module is itself a contribution: a reusable, driver-independent bridge between DPDK’s userspace memory model and the Linux DMA API for non-coherent ARM platforms, described in full in Section 4.3.

Status and Scope

This design was brought to a functional but not fully validated state. The PMD initialises the GEM hardware and establishes link, and the cache-synchronisation module has been verified against a coherent-memory fallback path, but the combined zero-copy datapath has not been benchmarked to the standard applied to the AF_XDP design in Chapter 5. It is reported here as a partial contribution and a concrete basis for future work, consistent with the exploratory objective and contribution set out in Chapter 1.

3.7 Summary

This chapter separated what already existed on this platform prior to this work (conventional sockets and DPDK/AF_XDP frameworks that are mature on x86 but entirely unsupported here) from the limitations that absence of driver and hardware support created (L1–L3, Table 3.1), and from the two designs developed in response. The primary contribution, native AF_XDP support in the `macb` driver, is architecturally complete and is the subject of the full performance evaluation in Chapter 5. The secondary contribution, a native DPDK PMD with explicit ARM cache synchronisation, confronts the harder zero-copy and cache-coherence problem (L3) directly, but is reported as a partial, exploratory result. Chapter 4 details the implementation of both designs.

Chapter 4

Implementation

4.1 Development Environment

The implementation and evaluation experiments were developed for Raspberry Pi 5 single-board computers running the 64-bit Raspberry Pi OS. The Raspberry Pi 5 systems use the Broadcom BCM2712 processor and RP1 I/O controller, with the Ethernet interface managed by the Cadence Gigabit Ethernet MAC (GEM). The Ethernet controller was accessed through the Linux `macb` driver, which was modified as part of this work to provide native XDP support. Two Raspberry Pi 5 systems were used for the networking experiments, connected through their Gigabit Ethernet interfaces to provide separate sender and receiver endpoints. The software components were developed primarily in C. The modified Linux kernel was based on version 6.18.

4.2 Adding Native XDP and AF_XDP Support in the Network Driver

The Raspberry Pi 5's Ethernet controller is exposed to Linux through the Cadence GEM (`macb`) network driver. Prior to this work, the driver provided the conventional Linux networking datapath but did not expose the native XDP operations required for an accelerated XDP/AF_XDP receive path. Consequently, AF_XDP could not make use of the driver-native datapath on the Raspberry Pi 5. The implementation described in this section extends the `macb` driver with the XDP interfaces required to execute eBPF programs on received packets, process the resulting XDP actions, transmit XDP frames, and register the driver's receive queues with the kernel XDP subsystem.

The implementation was confined primarily to the receive and transmit datapaths of the GEM driver. The existing DMA descriptor rings and page-pool based buffer management were retained, with the XDP implementation operating on the same packet buffers before they are converted into Linux `sk_buffs`. This is important because it allows packets accepted by an XDP program to continue through the normal networking stack, while packets redirected to an `AF_XDP` socket can leave the conventional path before an `sk_buff` is constructed.

Integration with the Linux XDP Interface

The first modification was to expose an XDP program attachment operation through the driver's `net_device` interface. The driver implements a `macb_xdp()` callback which accepts the kernel's `netdev_bpf` commands. XDP is restricted to GEM-capable instances of the driver, and the implementation additionally verifies that the configured receive buffer can fit within a single memory page when an XDP program is attached. This constraint is required because the implemented XDP receive path constructs an `xdp_buff` directly from the page allocated for the received frame.

The attached BPF program is stored in the driver's `xdp_prog` field using RCU-protected assignment. This allows the receive path to retrieve the currently active program without introducing a conventional lock into the per-packet processing path. The corresponding program reference is released when it is replaced.

The driver also registers each GEM receive queue with the XDP subsystem. For each queue, `xdp_rxq_info_reg()` maps the queue to the network device and its NAPI context, while `xdp_rxq_info_reg_mem_model()` specifies `MEM_TYPE_PAGE_POOL` as the memory model. If registration of any queue fails, previously registered queues are unregistered to prevent partial initialisation.

The resulting relationship between the driver and the kernel XDP subsystem can be summarised as outlined in Algorithm 1 (Appendix 7.1).

This registration does not replace the existing GEM receive mechanism. Instead, it introduces an XDP processing stage into the existing pipeline. Hence, packets can still follow the conventional `sk_buff` path when the XDP program returns `XDP_PASS`.

Modification of the Receive Path

The principal change to the driver is in `gem_rx()`, which processes completed receive descriptors during NAPI polling. The original receive path first determines whether the

DMA descriptor has been consumed by the hardware, retrieves the corresponding page, synchronises the page for CPU access, and then constructs an `sk_buff`. The modified implementation inserts XDP processing between DMA synchronisation and `sk_buff` construction.

At the beginning of NAPI processing, the driver obtains the currently attached BPF program using an RCU read-side critical section. For each completed descriptor, the driver verifies the descriptor state and retrieves the page associated with the receive buffer. The page is then synchronised for CPU access using the page-pool DMA synchronisation operation.

An `xdp_buff` is subsequently initialised using the page associated with the received packet. The XDP packet data begins at an offset determined by `gem_rx_page_offset()`, which is defined as `XDPPACKETHEADROOM + NET_IP_ALIGN`. The allocation size is therefore adjusted to accommodate the XDP headroom in addition to the receive buffer.

The packet is then passed directly to the attached BPF program through `bpf_prog_run_xdp()`. The program's return value determines whether the packet continues through the conventional stack or follows one of the XDP fast paths. The final receive algorithm is outlined in Algorithms 2 and 3 (Appendix 7.1).

The individual XDP actions provide the interface between the driver and the rest of the XDP subsystem. For instance, `XDPPASS` returns control to the pre-existing GEM receive implementation. The modified packet length and headroom are obtained from the `xdp_buff` before an `sk_buff` is constructed. The normal receive path then builds the socket buffer, marks it for page-pool recycling, performs protocol classification and checksum handling, and delivers it through `napi_gro_receive()`.

The remaining actions avoid construction of an `sk_buff`. For `XDPPDROP`, the page is promptly returned through `xdp_return_buff()`. For `XDPTX`, the received buffer is converted into an XDP frame and transmitted directly through the driver's XDP transmit path. For `XDPRREDIRECT`, the driver invokes `xdp_do_redirect()`, allowing the kernel XDP infrastructure to determine the packet's destination. Successful redirects are accumulated and flushed once the receive batch has been processed.

This ordering is the key difference between the modified driver and the original socket-based receive path. A packet directed towards an XDP consumer does not require construction of an `sk_buff`, protocol processing, or delivery through the normal socket receive path before it can be redirected.

RX Buffer Management and DMA Integration

Introducing XDP into the receive path required the existing page-pool allocation mechanism to be compatible with the XDP buffer model. The driver already uses pages obtained from a `page_pool` for receive buffers. The modified refill operation retains this mechanism while placing the DMA address at the XDP-compatible packet offset.

When an RX descriptor requires a new page, the driver acquires the page through `page_pool_dev_alloc_pages()`, records it in the queue's `rx_buffers_page` array, and fetches its DMA address through `page_pool_get_dma_addr()`. The address programmed into the GEM descriptor is offset by `gem_rx_page_offset()`, thereby leaving the required XDP headroom before the packet data. Descriptor writes are ordered with DMA write barriers before the descriptor is made available to the hardware.

This allows the same physical page to serve as the DMA receive buffer and the backing memory of the `xdp_buff`. Hence, The driver does not have to introduce a second packet buffer or copy the packet just to execute the XDP program. The corresponding buffer lifecycle can be represented as shown in Algorithm 4 (Appendix 7.1).

The use of the existing page-pool DMA mappings is particularly important for the XDP_TX path. Since an RX page is already associated with a DMA mapping, the driver can recover its DMA address through `page_pool_get_dma_addr()` rather than establishing a second mapping. The implementation consequently distinguishes between frames that originate from an existing RX page and frames that require a new DMA mapping.

XDP Transmit Path

XDP transmission requires a transmit path separate from the conventional `sk_buff`-based `macb_start_xmit()` implementation. The modified driver therefore introduces a dedicated XDP transmit queue, selected as the final GEM queue:

$$Q_{\text{XDP}} = Q_{N-1} \quad (4.1)$$

where N is the number of configured GEM queues. The helper `macb_xdp_tx_queue()` implements this selection.

The common XDP transmit operation is implemented by `macb_xdp_queue_one()`. Before placing a frame into the TX ring, the function validates that the frame does not contain unsupported fragments, that its length is valid, and that sufficient descriptor

space is available. It then records the XDP frame in the driver's TX bookkeeping structure.

Two DMA paths are supported. When a frame requires a new mapping, the driver uses `dma_map_single()` with `DMA_TO_DEVICE`. For an XDP frame originating from a page already managed by the RX page pool, the existing DMA address is recovered from the page-pool mapping instead. This avoids unnecessarily unmapping and remapping the same physical page. The resulting descriptor submission algorithm is detailed in Algorithm 5 (Appendix 7.1).

After frames have been queued, `macb_xdp_kick_tx()` starts transmission by setting the GEM `TSTART` control bit. The helper also handles the existing TX-pending state and low-power idle wake-up mechanism under the driver's hardware lock.

The same queueing primitive is used for both explicit XDP transmission and `XDPTX`. For `XDPTX`, the received `xdp_buff` is first converted into an `xdp_frame`; the frame is then queued using the existing RX page's DMA mapping. Explicit XDP transmission through the driver's `ndo_xdp_xmit` implementation can instead submit a batch of XDP frames, with the driver serialising access to the transmit queue using `tx_ptr_lock`.

Interaction with AF_XDP

The driver-side modifications provide the functionality required for the kernel XDP infrastructure to use the GEM receive queues as XDP-capable queues. In particular, the driver now provides the complete XDP action processing path and implements the `XDPRIRECT` operation through `xdp_do_redirect()`. This redirect mechanism is the interface through which an XDP packet can subsequently be delivered to an `AF_XDP` socket.

The resulting receive path for an `AF_XDP` packet is therefore:

```
GEM DMA → page-pool buffer → xdp_buff → BPF/XDP program →
XDP_REDIRECT → AF_XDP socket/UMEM
```

rather than:

```
GEM DMA → page → sk_buff → GRO → TCP/IP → socket receive queue.
```

The driver therefore removes the normal `sk_buff` construction and higher-level network-stack processing for packets redirected by XDP. The uploaded implementation registers the RX queues using the page-pool memory model and exposes the XDP

redirect operation, while the AF_XDP socket and UMEM management remains part of the Linux XDP subsystem.

This distinction is important when interpreting the implementation as “native AF_XDP support”. The work does not reimplement the AF_XDP socket subsystem. Instead, it adds the missing driver-level XDP functionality that allows the existing Linux AF_XDP infrastructure to operate on the Cadence GEM device.

Together, these modifications insert the XDP processing stage directly into the GEM receive path while retaining the existing Linux networking path for packets returned with XDP_PASS. The implementation consequently supports both conventional kernel networking and the accelerated XDP redirect path on the same network device, providing the driver functionality required for subsequent AF_XDP evaluation.

4.3 Alternative DPDK PCI Poll-Mode Driver

As part of the alternative design and future work for this dissertation, a prototype DPDK Poll Mode Driver (PMD) for the Cadence Gigabit Ethernet (GEM) controller was also developed. This implementation specifically targets the **Raspberry Pi 5**, where the GEM controller is integrated into the **RP1 I/O controller** and exposed to the host system via the PCIe bus.

Implementation Architecture

The driver is structured as a standard DPDK PCI PMD, following the `librte_ethdev` framework. It comprises two primary components:

- **Ethernet Device Logic:** Handles the PCI probe/remove lifecycle, device initialization, and configuration of the GEM core. It implements the `eth_dev_ops` interface for basic port control (start, stop, statistics) and leverages the RP1-specific memory map where the GEM registers are located at a fixed offset within the PCI BAR1 address space.
- **Rx/Tx Datapath:** Implements the core descriptor-based DMA rings. The driver uses standard `rte_mbuf` structures for packet buffers and manages the Cadence-specific DMA descriptor format (consisting of an address word and a control word). The data path includes basic support for packet transmission (`macb_xmit_pkts`) and reception (`macb_recv_pkts`), including necessary memory barriers for hardware synchronization.

Key Features and Progress

RP1 Peripheral Mapping The driver correctly identifies the GEM core within the RP1's Proc-address map, using inferred offsets from the hardware documentation to access GEM and ETH_CFG registers.

MDIO and PHY Support Basic IEEE 802.3 Clause 22 MDIO support was implemented to scan for and initialize the external PHY over the RGMII interface, ensuring the hardware link is active before data transfer begins.

Hardware Compatibility The driver supports the GEM hardware features required for standard Ethernet operation, such as RGMII interface selection, gigabit speed configuration, and FCS discard.

PCI Binding The implementation is compatible with standard DPDK userspace I/O drivers like `uio_pci_generic` and `vfio-pci`, allowing the controller to be bound to DPDK applications such as `testpmd`.

While the current implementation provides a functional foundation for packet I/O, it serves as a proof-of-concept for high-performance networking on ARM-based Single Board Computers (SBCs).

Cache Synchronisation Kernel Module

The `cache_mem_sync` kernel module provides a specialized helper designed to facilitate explicit cache maintenance for userspace DMA drivers on non-coherent ARMv8 systems, such as the Raspberry Pi 5. This module is intended for use in conjunction with a custom DPDK PCI Poll Mode Driver (PMD) for the Cadence GEM (`macb`) Ethernet controller.

Architectural Overview

The module implements a character device interface (`/dev/cache_mem_sync`) that allows userspace applications to request privileged cache operations. This approach bridges the gap between DPDK's userspace memory management and the Linux kernel's DMA subsystem.

Key Implementation Details

- **Memory Pinning and Registration:** Upon registration (`CACHE_MEM_SYNC_REGISTER`), the module pins userspace memory pages using `get_user_pages_fast()`. This ensures that the buffers remain resident in physical memory during DMA transactions.
- **Device-Specific DMA Mapping:** Through the `CACHE_MEM_SYNC_SET_DEV` ioctl, the helper binds to the specific `macb` platform or PCI device. This association is critical for ensuring that cache maintenance operations utilize the correct DMA mask and attributes associated with the hardware.
- **Synchronisation Datapath:** The module implements ownership transitions via `SYNC_TO_DEVICE` and `SYNC_FROM_DEVICE`. Internally, these map to the Linux kernel's DMA API (`dma_sync_single_for_device` and `dma_sync_single_for_cpu`), performing the necessary cache cleans and invalidations required for non-coherent ARMv8 architectures.
- **Coherent Buffer Fallback:** For validation purposes, the implementation includes a fallback mechanism that utilizes `dma_alloc_coherent`. This provides a reference point for verifying cache behaviour by simulating hardware writes and ensuring they are correctly observed by userspace after synchronization.

DPDK Integration Flow

In the proposed architecture, the DPDK PMD registers its hugepage-backed mbuf pools with the `cache_mem_sync` module during initialization. During packet transmission, the PMD issues a synchronization call before the NIC initiates DMA reading. Conversely, during packet reception, the PMD triggers a cache invalidation after the NIC has written to the buffer, ensuring data consistency for the Application layer.

Chapter 5

Evaluation

5.1 Experimental Setup

The experiments were performed using two Raspberry Pi 5 computers running Raspberry Pi OS with a Linux 6.18 kernel. The kernel on both systems was modified to incorporate the custom `macb` Ethernet driver described in the previous chapter. The two Raspberry Pi systems were connected via their onboard Ethernet interfaces using a 1000 Mbps Ethernet link, with one system acting as the sender/client and the other as the receiver/server. This provided a consistent hardware and software environment for evaluating the different packet-processing approaches.

A dedicated networking benchmark suite was developed in C to provide a common framework for the experiments. The suite implements TCP and UDP benchmarks using the conventional Linux socket interface, alongside DPDK benchmarks using the `AF_XDP` PMD. It measures network throughput, packets per second, packet counts, and transmission/reception errors, while the latency benchmarks additionally record minimum, average, maximum, and percentile latency values. The benchmarks support varying packet payload sizes and automated sweeps, with results stored in a common CSV format to facilitate direct comparison between the different networking implementations.

5.2 Round-Trip Latency (RTT) Performance

Round-trip-time (RTT) latency was measured for UDP, TCP, and DPDK using packet sizes from 64 to 1400 bytes, with 100,000 samples collected for each configuration. The complete raw results are provided in Table 7.1 (Appendix 7.2). Figure 5.1 presents

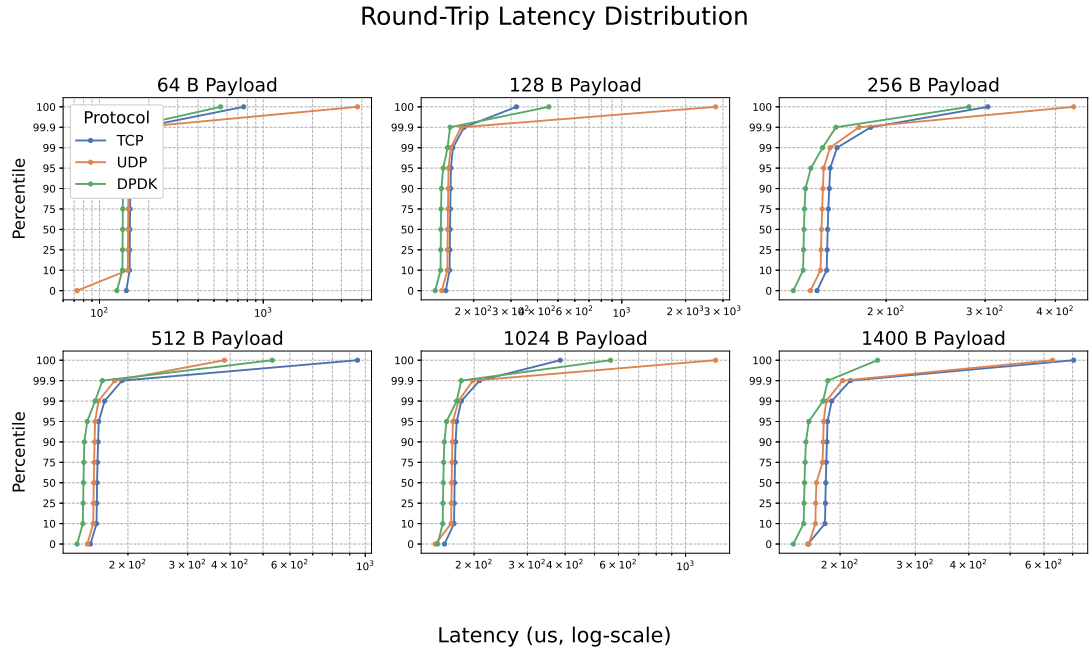


Figure 5.1: Round-Trip Latency Comparison of TCP, UDP, and DPDK for varying different payload sizes

the median and tail-latency results across the tested packet sizes, while Table 5.1 summarises the median RTT and the improvement provided by DPDK.

Table 5.1: Median RTT and DPDK improvement by packet size.

Packet size (bytes)	UDP (μ s)	TCP (μ s)	DPDK (μ s)	vs. UDP (%)	vs. TCP (%)
64	149.463	153.334	138.519	7.32	9.66
128	150.852	154.648	140.000	7.19	9.47
256	153.482	157.425	142.834	6.94	9.27
512	158.666	162.278	147.945	6.76	8.83
1024	168.537	172.389	158.130	6.17	8.27
1400	176.037	185.167	165.129	6.20	10.82
Mean	159.506	164.207	148.760	6.74	9.41

As shown in Figure 5.1 and Table 5.1, DPDK achieved the lowest median RTT at every packet size. Averaged across all packet sizes, its median RTT was 148.7 μ s, compared to 159.5 μ s for UDP and 164.2 μ s for TCP. DPDK therefore reduced median latency by 6.7% relative to UDP and 9.4% relative to TCP. Across individual packet sizes, the improvement ranged from 6.1% to 7.3% against UDP and from 8.2% to 10.8%

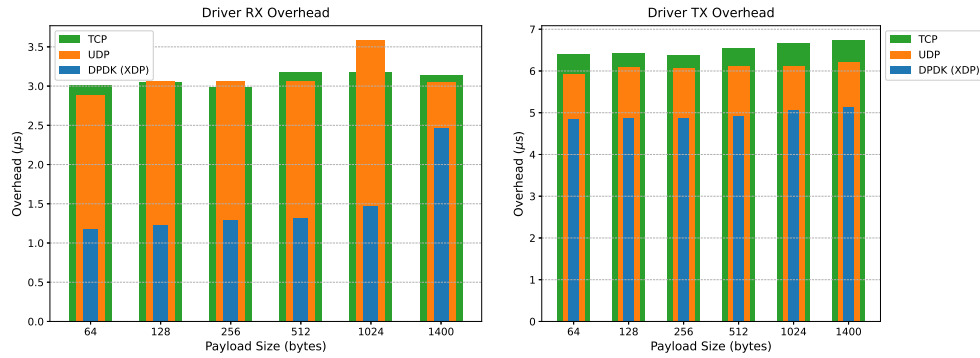


Figure 5.2: Receive/Transmit Driver Overhead Comparison for TCP, UDP, and XDP data paths.

against TCP.

RTT also increased with packet size for all three test types, but DPDK maintained its advantage throughout the tested range. The tail-latency charts in Figure 5.1 also show lower high-percentile (i , p99) latency for DPDK. Averaged across the packet sizes, DPDK recorded a p99.9 RTT of $167.85 \mu\text{s}$, compared to $185.39 \mu\text{s}$ for UDP and $193.49 \mu\text{s}$ for TCP. This represents a reduction of 9.46% relative to UDP and 13.25% relative to TCP.

Overall, the results indicate that DPDK improves both typical and high-percentile RTT latency. This improvement is consistent with DPDK’s user-space packet-processing and kernel-bypass design, which reduces the overheads associated with the conventional networking stack [7]. Under the tested conditions, the results show a consistent latency ordering of DPDK, UDP, and TCP.

5.3 Driver Overhead

Figure 5.2 presents the time spent in the driver transmit (TX) and receive (RX) paths for TCP, UDP, and DPDK/XDP across payload sizes from 64 to 1400 bytes. The measurements were collected using `bpftrace` and are reported in microseconds. The corresponding raw values are provided in Table 7.2 (Appendix 7.2). Across all payload sizes, XDP incurred the lowest measured overhead in both directions. Mean TX overhead was $4.95 \mu\text{s}$ for XDP, compared with $6.09 \mu\text{s}$ for UDP and $6.52 \mu\text{s}$ for TCP. XDP therefore reduced mean TX overhead by 18.7% relative to UDP and 24.1% relative to TCP.

TX overhead increased only moderately with payload size. Between 64 and 1400

bytes, TCP increased from 6.39 to 6.74 μ s, UDP from 5.93 to 6.21 μ s, and XDP from 4.85 to 5.14 μ s. These increases are small compared with the change in payload size, indicating that the measured TX cost was dominated by fixed per-packet operations rather than work proportional to the number of bytes. The ordering remained consistent at every payload size, with XDP exhibiting the lowest overhead, followed by UDP and then TCP.

The advantage of XDP was more pronounced in the RX path. Mean RX overhead was 1.493 μ s for XDP, compared with 3.09 μ s for TCP and 3.12 μ s for UDP, corresponding to reductions of approximately 51.7% and 52.1%, respectively. TCP RX overhead remained between 2.98 and 3.17 μ s, while UDP was similarly stable except for an isolated increase to 3.59 μ s at 1024 bytes. For payloads up to 1024 bytes, XDP RX overhead increased gradually from 1.17 to 1.47 μ s and remained approximately 54–61% lower than TCP and 57–60% lower than UDP.

At 1400 bytes, XDP RX overhead rose sharply to 2.46 μ s, reducing its advantage over TCP and UDP to approximately 21.4% and 19.4%, respectively. This discontinuity may indicate a size-dependent threshold in buffer, DMA, cache, or descriptor handling, although repeated trials and measures of dispersion would be required to distinguish such an effect from experimental variation. Overall, the results indicate that XDP substantially reduces driver-resident processing time, particularly during reception. However, because these measurements cover only the instrumented driver paths, they should not be interpreted directly as end-to-end application latency.

5.4 Throughput Performance

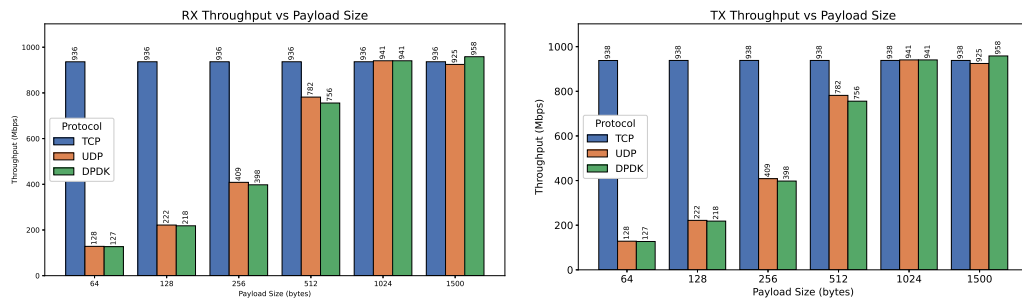


Figure 5.3: Throughput comparison of TCP, UDP, and DDPK for varying different payload sizes

Transmit (TX) and receive (RX) throughput were measured for TCP, UDP, and

DPDK using packet sizes from 64 to 1500 bytes over 30-second test periods. The complete raw results are provided in Table 7.3 (Appendix 7.2). Figure 5.3 shows the throughput trends across packet sizes, while Table 5.2 summarises the measured TX and RX throughput.

Table 5.2: TX and RX throughput by packet size.

Packet size (bytes)	TCP TX (Mbps)	UDP TX (Mbps)	DPDK TX (Mbps)	TCP RX (Mbps)	UDP RX (Mbps)	DPDK RX (Mbps)
64	938.065	128.325	127.006	936.263	128.330	127.006
128	938.241	221.648	218.382	936.413	221.640	218.382
256	938.323	408.529	397.828	936.419	408.520	397.828
512	938.262	781.738	755.758	936.450	781.710	755.758
1024	938.339	940.723	940.721	936.450	940.713	940.697
1500	938.271	924.727	958.466	936.443	924.717	958.466

As shown in Figure 5.3, TCP maintained almost constant throughput across all application payload sizes, with TX throughput of approximately 938 Mbps and RX throughput of approximately 936 Mbps. This behaviour results from TCP’s byte-stream semantics and the functionality of the TCP/IP stack. Application writes do not necessarily correspond one-to-one with transmitted packets because the kernel can buffer and coalesce data before segmenting the stream according to the maximum segment size [4]. Segmentation-offload mechanisms may further allow the kernel and NIC to process larger buffers efficiently. TCP can therefore maintain near-link throughput even when the application supplies data using small writes.

In contrast, UDP and DPDK throughput grew with packet size as their performance was more strongly affected by fixed per-packet processing costs. DPDK TX throughput increased from 127.01 Mbps at 64 bytes to 958.46 Mbps at 1500 bytes. It performed slightly below UDP between 64 and 512 bytes, was approximately equal to UDP at 1024 bytes, and achieved the highest throughput at 1500 bytes. At this payload size, DPDK exceeded UDP by 33.73 Mbps (3.65%) and TCP by 20.19 Mbps (2.15%) in the TX test. Its RX throughput of 958.466 Mbps similarly exceeded UDP by 33.74 Mbps (3.65%) and TCP by 22.02 Mbps (2.35%). This demonstrates that DPDK provides a throughput improvement at larger payload sizes and was the only tested implementation to exceed 950 Mbps.

A flatter DPDK throughput profile, similar to TCP, could be achieved by increasing

small-packet processing capacity. Relevant optimisations include sending and receiving packets in larger bursts, reducing work within the polling loop, reusing preallocated packet buffers, using dedicated CPU cores and NUMA-local memory, and distributing traffic across multiple queues and cores. Where application requirements permit, aggregating data into larger frames or using jumbo frames would further reduce the effect of fixed per-packet costs. TCP's constant throughput should nevertheless not be interpreted as a direct packet-for-packet advantage, since its stack may coalesce small application writes into larger transmitted segments.

5.5 Packet-Processing Rate

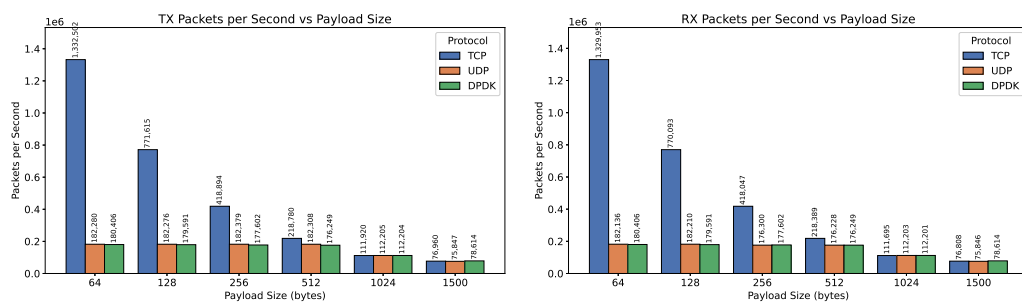


Figure 5.4: Packets per second comparison of TCP, UDP, and DDPK for varying different payload sizes

The packet-processing rate was measured for TCP, UDP, and DDPK using packet sizes from 64 to 1500 bytes. The complete results are provided in Table 7.4 (Appendix 7.2), while Figure 5.4 shows the transmit (TX) and receive (RX) rates across the tested packet sizes. TCP recorded its highest rate at the smallest application write size, decreasing from approximately 1.33 million operations per second at 64 bytes to approximately 0.077 million operations per second at 1500 bytes. In comparison, UDP and DDPK remained near 0.18 million packets per second between 64 and 512 bytes before decreasing to approximately 0.112 million packets per second at 1024 bytes and 0.08 million packets per second at 1500 bytes.

TCP's high reported rate for small application writes should not be interpreted directly as the number of Ethernet frames transmitted per second. TCP provides a byte stream and does not preserve application write boundaries. The TCP/IP stack can buffer and combine multiple writes into larger TCP segments, potentially assisted by segmentation-offload mechanisms. Consequently, the reported TCP value of approxi-

mately 1.33 million operations per second at 64 bytes primarily represents the rate of application-level send or receive completions rather than a like-for-like physical packet rate. Direct comparison with UDP datagrams and DPDK frames must therefore be treated cautiously.

As shown in Figure 5.4, UDP and DPDK sustained approximately 176,000–182,000 packets per second between 64 and 512 bytes, indicating a small-packet processing limit in the tested implementations. DPDK TX performance was between 1.03% and 3.32% below UDP over this range. However, UDP’s RX rate was approximately 3.33% lower than its TX rate at 256 and 512 bytes, whereas DPDK produced nearly identical TX and RX rates. This suggests that some UDP datagrams were not received at these sizes, while DPDK maintained more consistent delivery.

At 1024 bytes, UDP and DPDK produced effectively identical rates of approximately 112,200 packets per second. At 1500 bytes, DPDK achieved the highest rate of 78,614 packets per second, exceeding UDP by 3.65% and the reported TCP rate by 2.15% in the TX results. The decline in packet rate for larger packets is expected as the link approaches its bit-rate capacity, since fewer large packets are needed to achieve near-gigabit throughput. Further improvements to DPDK’s small-packet rate could be obtained through larger processing bursts, packet-buffer reuse, dedicated CPU cores, NUMA-local memory, and multi-queue scaling.

5.6 Discussion

Taken together, the four experiments point to a consistent, but not unconditional, benefit from the XDP-based data path. In latency and driver overhead, DPDK/XDP was strictly better than both conventional protocols at every payload size and at both the median and the tail, and the two experiments agree in ordering: lower driver-resident processing time (Section 5.3) coincides with lower RTT (Section 5.2) across the same range of conditions. This agreement, obtained from independent measurement methods, supports the conclusion that shortening the kernel’s involvement in the receive and transmit paths is genuinely responsible for the observed latency improvement, rather than an artefact of the measurement technique. The disparity in scale between the two, a large reduction in driver time versus a comparatively modest reduction in RTT, is expected rather than contradictory, since `bpfttrace` measures only the instrumented driver functions, while RTT accumulates transmission, application processing, scheduling, and link delay on top of that.

Throughput and packet rate tell a different, workload-dependent story. TCP's near-constant, near-line-rate throughput is a product of stream coalescing and segmentation offload rather than evidence of a lighter per-packet path, and is not directly comparable to UDP's or DPDK's per-datagram behaviour; the fairer comparison is between UDP and DPDK, where DPDK trails at small payloads but overtakes UDP, and every implementation, once payloads are large enough to approach link capacity. This pattern is consistent with fixed per-packet software costs, rather than link bandwidth, being the binding constraint at small payload sizes, and is not contradicted by DPDK's lower driver overhead, since end-to-end throughput and packet rate depend on the whole processing pipeline (batching, buffer management, queueing, and application-loop efficiency) and not on driver time alone.

The overall picture is therefore one of a targeted rather than blanket improvement. The evaluated AF_XDP path delivers a clear, consistent advantage in latency, tail latency, and driver overhead at every tested payload size, and the strongest combined result at large payloads, where it also leads on throughput and packet rate. It does not yet improve small-packet throughput or packet rate over conventional UDP, indicating that the current implementation benefits from a shorter data path but has not yet been tuned to exploit DPDK's small-packet processing capacity, a gap Section 6.1 returns to.

5.7 Limitations

These results should be read within the bounds of a controlled, single-link setup. Two Raspberry Pi 5 systems connected directly over a single 1 Gbps Ethernet link, with one sender and one receiver, using the specific kernel, driver, and AF_XDP PMD implementation described in Chapter 4. This improves repeatability but leaves scaling behaviour across multiple flows, queues, cores, or network hops, along with generalisation to other hardware or kernel versions, untested.

The driver-overhead measurements in particular carry methodological caveats beyond this scope. The instrumentation itself adds probe and aggregation overhead that may be non-negligible at the few-microsecond scale being measured, and the TCP/UDP and XDP probe boundaries do not cover strictly equivalent amounts of work, so absolute values should be treated as indicative rather than exact. No experiment in this chapter was repeated across independent runs or reported with dispersion measures such as confidence intervals; consequently, the isolated deviations noted in Sections 5.3 and 5.6 cannot be confidently distinguished from ordinary run-to-run variation, and all reported

figures should be read as single-run driver-resident execution times and throughput/rate values rather than statistically validated estimates.

Finally, the evaluation is limited to RTT, throughput, packet rate, and driver execution time; it does not measure CPU utilisation, cycles per packet, power consumption, memory bandwidth, or behaviour under bidirectional or multi-flow traffic. This matters particularly for the latency results, since DPDK's poll-mode operation trades reduced latency for a continuously occupied CPU core, a cost the current results do not quantify. Section 6.1 outlines the repeated-trial, instrumentation, and workload extensions needed to close these gaps.

Chapter 6

Conclusion

6.1 Future Research Directions

The most immediate direction for future work is to complete and validate the native DPDK PMD prototyped in this dissertation. It currently implements PCI BAR mapping, GEM initialisation, and descriptor-ring management, but its zero-copy datapath and the `cache_mem_sync` module require further testing and validation. A validated PMD would enable a direct three-way comparison between conventional sockets, AF_XDP, and native DPDK on identical hardware, isolating the cost of kernel participation from that of direct userspace device access.

A second direction is extending the modified `macb` driver from its current one-copy AF_XDP design to true zero-copy operation, via native UMEM registration and queue management that let the GEM DMA engine transfer packets directly into application-owned buffers. This would remove the remaining receive-side copy, at the cost of additional DMA mapping and cache-synchronisation complexity, and would show whether copy elimination or queue/cache handling dominates on this platform.

The evaluation also points to a small-packet ceiling worth optimising directly – DPDK-via-AF_XDP plateaued at approximately 180,000 packets per second and trailed UDP below 512 B. Sweeping burst size, ring depth, polling strategy, CPU and interrupt affinity, and multi-queue scaling would help isolate whether this ceiling originates in the application, the AF_XDP rings, the driver, or the controller itself.

Further experiments should strengthen the statistical and architectural coverage of these results. Each configuration should be extended with CPU cycles per packet, cache-miss, and energy measurements. These additional metrics are particularly important for wimpy-node platforms, where polling trades reduced latency for continuous CPU

occupancy. The sharp increase in AF_XDP RX overhead near 1400 B (Section 5.3) warrants targeted follow-up, alongside testing at higher link speeds, under bidirectional traffic, and across larger Raspberry Pi clusters.

Finally, future work should integrate this data path into a real distributed edge workload rather than evaluating networking in isolation to determine whether the measured reductions in RTT and driver overhead translate into improved job completion time, tail response time, and cluster energy efficiency. Candidates for such workloads could include distributed matrix multiplication, edge inference, key-value serving, and lightweight task scheduling.

6.2 Final Remarks

This dissertation asked whether kernel-bypass networking can benefit low-power ARM edge hardware. The Raspberry Pi 5's Cadence GEM controller had neither native XDP support nor a DPDK PMD. The primary contribution closes this gap by extending the `macb` driver with native XDP execution, action handling, page-pool-compatible buffers, and a dedicated XDP transmit path, enabling AF_XDP and DPDK's AF_XDP PMD on this hardware for the first time.

The results support a clear, bounded claim. DPDK-over-AF_XDP reduced median RTT by 9.41% against TCP and 6.74% against UDP, with a larger tail-latency benefit (13.25% and 9.46% at p99.9). Driver-level tracing attributes this to reduced software overhead: 24.1% lower TX cost than TCP and $\sim 52\%$ lower RX cost than both baselines. Throughput, however, tells a more nuanced story. TCP's stream semantics and offloads sustained near-link throughput throughout, while DPDK trailed UDP below 512 B but overtook both at 1500 B (the only path to exceed 950 Mbps). Kernel bypass, in other words, is not a uniform win; its advantage is concentrated in latency, tail behaviour, and large-packet transfer, not blanket throughput improvement.

Kernel bypass will not replace the conventional stack on the Raspberry Pi 5, and should not. TCP remains the right choice wherever protocol functionality matters more than latency. However, what this work shows is that a shorter, specialised packet path is achievable on constrained ARM hardware, and where latency and driver overhead are the bottleneck, it delivers a measurable, repeatable benefit. Closing this driver-support gap turns kernel-bypass networking from a server-class technique into an option available at the edge.

Bibliography

- [1] David G Andersen, Jason Franklin, Michael Kaminsky, Amar Phanishayee, Lawrence Tan, and Vijay Vasudevan. Fawn: A fast array of wimpy nodes. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 1–14, 2009.
- [2] David G Andersen, Jason Franklin, Michael Kaminsky, Amar Phanishayee, Lawrence Tan, and Vijay Vasudevan. Fawn: A fast array of wimpy nodes. *Communications of the ACM*, 54(7):101–109, 2011.
- [3] Giovanni Bartolomeo, Mario Yosofie, Simon Baurle, Oliver Haluszczynski, Nitinder Mohan, and Jörg Ott. Oakestra: A lightweight hierarchical orchestration framework for edge computing. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 215–231, 2023.
- [4] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. Understanding host network stack overheads. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, SIGCOMM ’21, page 65–77, New York, NY, USA, 2021. Association for Computing Machinery.
- [5] Ruining Chen and Guoao Sun. A survey of kernel-bypass techniques in network stack. In *Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence*, pages 474–477, 2018.
- [6] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Commun. ACM*, 51(1):107–113, January 2008.
- [7] DPDK Project. Af_xdp poll mode driver. https://doc.dpdk.org/guides/nics/af_xdp.html. Data Plane Development Kit documentation. Accessed 2026.

- [8] Marco Fogli et al. Comparative analysis of lightweight kubernetes distributions for edge computing: Performance and resource efficiency. In *Service-Oriented and Cloud Computing, Communications in Computer and Information Science*, 2025.
- [9] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 281–297, 2020.
- [10] DPDK Intel. Data plane development kit, 2014.
- [11] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for μ second-scale tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 345–360, 2019.
- [12] Dumitrel Loghin, Lavanya Ramapantulu, and Yong Meng Teo. On understanding time, energy and cost performance of wimpy heterogeneous systems for edge computing. In *2017 IEEE International Conference on Edge Computing (EDGE)*, pages 1–8. IEEE, 2017.
- [13] Chirag Modi and Sameer G Kulkarni. Kernel bypass and user-space network frameworks for high-performance computing workloads. In *2024 IEEE 31st International Conference on High Performance Computing, Data and Analytics Workshop (HiPCW)*, pages 187–188. IEEE, 2024.
- [14] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high cpu efficiency for latency-sensitive datacenter workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 361–378, 2019.
- [15] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. Sparrow: distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP)*, pages 69–84, 2013.
- [16] Kyle A. Simpson, Chris Williamson, Douglas J. Paul, and Dimitrios P. Pezaros. GALETTE: a lightweight XDP dataplane on your Raspberry Pi. In *IFIP Networking 2023*, Barcelona, Spain, 2023.

- [17] Rachel Somerset and Yuvraj Patel. A distributed framework for matrix multiplication on a cluster raspberry pis. In *MobiUK 2025*, 2025.
- [18] The Linux Kernel Documentation. Af_xdp. https://docs.kernel.org/networking/af_xdp.html. Accessed 2026.
- [19] Shuo Wang, Xing Zhang, Yan Zhang, Lin Wang, Juwo Yang, and Wenbo Wang. A survey on mobile edge networks: Convergence of computing, caching and communications. *IEEE Access*, 5:6757–6779, 2017.

Chapter 7

Appendix

7.1 Algorithms

Input: GEM network device, optional BPF program P

Output: XDP program and receive queues registered with the kernel

if *device is not GEM* **then**

return $-EOPNOTSUPP$;

end

if P *is being attached* **then**

if *device is running and RX allocation size exceeds one page* **then**

 Reject attachment;

return;

end

end

Retrieve currently attached program P_{old} ;

Atomically replace P_{old} with P ;

Update BPF program reference counts;

foreach *RX queue* q **do**

 Register q with the XDP subsystem;

 Associate q with its page-pool memory model;

end

return success;

Algorithm 1: XDP Program Registration

Input: GEM RX descriptor ring and optional XDP program P

Output: Received frame processed by XDP or normal networking stack

Enter RCU read-side critical section and Retrieve attached XDP program P ;

```

while NAPI budget remains do
    Read current RX descriptor;
    if descriptor is not marked RX_USED then
        | break;
    end
    Synchronise descriptor and packet metadata;
    Retrieve page and remove it from RX buffer tracking;
    Synchronise page for CPU access;
    Determine packet length;
    if  $P$  is attached then
        | Initialise xdp_buff using RX page;
        |  $a \leftarrow$  execute XDP program  $P$ ;
        | Process XDP action  $a$  using Algorithm 3;
        | if  $a \neq$  XD_PASS then
        | | continue;
        | end
    end
    Construct sk_buff from received page;
    Pass packet to Linux networking through GRO;
end
if redirects are pending then
    | Flush XDP redirects;
end
Replenish RX descriptors;
Leave RCU read-side critical section;

```

Algorithm 2: GEM Receive Processing with XDP

Input: XDP action a , received XDP buffer

Output: Action-specific buffer handling

switch a **do**

case XDP_PASS **do**

 Update packet length and headroom;

return;

end

case XDP_DROP **do**

 Return XDP buffer to page pool;

 Update drop statistics;

end

case XDP_TX **do**

 Transmit XDP frame through GEM TX queue;

 Return buffer on failure;

end

case $XDP_REDIRECT$ **do**

 Redirect packet through XDP mechanism;

 Mark redirect flush as pending;

end

case $XDP_ABORTED$ **do**

 Record XDP exception;

 Return XDP buffer;

 Update error statistics;

end

end

Algorithm 3: XDP Action Processing

Input: GEM RX descriptor ring

Output: Received buffer processed through the XDP lifecycle

Allocate a page from the page pool;

Obtain the DMA address associated with the page;

Add XDP packet headroom to the DMA address;

Program the RX descriptor with the buffer address;

Issue a DMA write barrier;

Enable GEM hardware to receive the packet;

GEM writes the packet into the page using DMA;

CPU observes the completed RX descriptor;

Synchronise the page for CPU access;

Construct `xdp_buff` over the received page;

Execute the XDP program;

if *packet is passed* **then**

 Construct `sk_buff` over the same page;

 Pass packet to the Linux networking stack;

end

else if *packet is dropped* **then**

 Return the page to the page pool for recycling;

end

else if *packet is redirected or transmitted* **then**

 Transfer ownership of the XDP buffer to the destination;

end

Replenish the RX descriptor ring;

Algorithm 4: XDP-Compatible RX Buffer Lifecycle

Input: XDP frame F , GEM TX queue Q

Output: F is submitted to the GEM TX ring

```

if  $F$  contains fragments then
    |   return -EOPNOTSUPP;
end

if  $F$  has invalid length then
    |   return -EINVAL;
end

if TX ring has fewer than two free descriptors then
    |   return -EBUSY;
end

Select current TX descriptor  $D$ ;
Store  $F$  in TX bookkeeping structure;
if frame requires DMA mapping then
    |    $M \leftarrow \text{dma\_map\_single}(F, \text{DMA\_TO\_DEVICE});$ 
    |   if mapping failed then
    |       |   return -ENOMEM;
    |   end
end

else
    |    $M \leftarrow \text{existing page-pool DMA address} + \text{page offset};$ 
end

Mark following descriptor as TX_USED;
Write DMA address  $M$  into  $D$ ;
Issue write memory barrier;
Set frame length and TX_LAST in  $D$ ;
Advance TX head pointer;
Update TX queue accounting;
return success;

```

Algorithm 5: Queue an XDP Frame for GEM Transmission

7.2 Raw Benchmark Results

Table 7.1: Raw round-trip latency percentiles (μ s) across protocol and payload size, corresponding to the latency distribution figures in Chapter 5.

Protocol	Payload (B)	Min	p10	p25	p50	p75	p90	p95	p99	p99.9	Max
TCP	64	145.8	152.8	153.0	153.3	153.8	154.3	154.9	158.4	180.9	759.6
TCP	128	147.7	154.0	154.3	154.6	155.1	155.6	156.1	159.6	180.4	318.0
TCP	256	150.7	156.8	157.1	157.4	157.8	158.4	159.0	163.6	187.8	303.4
TCP	512	155.2	161.7	161.9	162.3	162.7	163.4	164.1	170.7	192.1	948.5
TCP	1024	159.6	171.6	172.0	172.4	172.9	173.6	175.1	181.4	208.4	385.1
TCP	1400	168.3	184.4	184.8	185.2	185.7	186.3	186.9	191.2	211.4	704.0
UDP	64	73.0	149.0	149.2	149.5	149.8	150.3	150.8	154.2	175.0	3763.5
UDP	128	141.4	150.3	150.6	150.9	151.2	151.7	152.2	155.9	174.5	2770.5
UDP	256	146.7	152.9	153.2	153.5	153.9	154.4	154.9	159.0	179.0	431.2
UDP	512	151.8	158.1	158.3	158.7	159.1	159.6	160.1	163.8	183.0	384.6
UDP	1024	148.7	167.9	168.2	168.5	168.9	169.5	170.1	177.5	198.1	1256.9
UDP	1400	169.0	175.1	175.4	176.0	181.9	182.6	183.1	185.6	202.7	628.5
DPDK	64	127.7	138.1	138.3	138.5	138.8	139.3	142.0	149.0	153.0	548.7
DPDK	128	131.6	139.5	139.7	140.0	140.3	140.9	143.6	150.6	154.7	452.4
DPDK	256	136.7	142.4	142.6	142.8	143.2	143.7	147.0	154.2	162.8	280.7
DPDK	512	141.7	147.4	147.6	147.9	148.3	148.9	151.7	160.0	168.2	532.1
DPDK	1024	151.5	157.4	157.7	158.1	158.6	159.4	162.1	175.1	181.3	565.2
DPDK	1400	155.4	164.4	164.7	165.1	165.6	166.4	169.0	182.4	187.2	244.7

Table 7.2: Raw packet processing overhead measurements for TCP, UDP, and XDP across different payload sizes. Values are reported in nanoseconds.

Payload (B)	TCP TX	TCP RX	UDP TX	UDP RX	XDP TX	XDP RX
64	6399	3015	5932	2888	4858	1176
128	6422	3050	6091	3062	4863	1228
256	6371	2987	6081	3068	4882	1291
512	6545	3179	6117	3060	4912	1321
1024	6678	3178	6116	3590	5059	1477
1400	6747	3135	6219	3054	5143	2463

Table 7.3: Raw throughput results (RX/TX) across protocol and payload size, corresponding to the RX/TX throughput figures in Chapter 5.

Protocol	Payload Size (B)	RX Throughput (Mbps)	TX Throughput (Mbps)
TCP	64	936.26	938.06
TCP	128	936.41	938.24
TCP	256	936.42	938.32
TCP	512	936.45	938.26
TCP	1024	936.45	938.34
TCP	1500	936.44	938.27
UDP	64	128.33	128.33
UDP	128	221.64	221.65
UDP	256	408.52	408.53
UDP	512	781.71	781.74
UDP	1024	940.71	940.72
UDP	1500	924.72	924.73
DPDK	64	127.01	127.01
DPDK	128	218.38	218.38
DPDK	256	397.83	397.83
DPDK	512	755.76	755.76
DPDK	1024	940.70	940.72
DPDK	1500	958.47	958.47

Table 7.4: Raw packets-per-second results (RX/TX) across protocol and payload size, corresponding to the RX/TX packets-per-second figures in Chapter 5.

Protocol	Payload Size (B)	RX (pkt/s)	TX (pkt/s)
TCP	64	1,329,953	1,332,502
TCP	128	770,093	771,615
TCP	256	418,047	418,894
TCP	512	218,389	218,780
TCP	1024	111,695	111,920
TCP	1500	76,808	76,960
UDP	64	182,136	182,280
UDP	128	182,210	182,276
UDP	256	176,300	182,379
UDP	512	176,228	182,308
UDP	1024	112,203	112,205
UDP	1500	75,846	75,847
DPDK	64	180,406	180,406
DPDK	128	179,591	179,591
DPDK	256	177,602	177,602
DPDK	512	176,249	176,249
DPDK	1024	112,201	112,204
DPDK	1500	78,614	78,614